
CASE STUDY / 2026

RouteOptim

Open-source delivery-route optimisation platform for small businesses.

Django 5 full-stack web application • Logistics & fleet management

PREPARED BY

Mudasir Shafiq

Full-stack & data engineer · chmudasir.fr@gmail.com

CONTENTS

Table of Contents

01	Project Overview	Page 3
02	System Architecture	Page 4
03	Data Model & Entity Relationships	Page 5
04	Technology Stack	Page 6
05	The Challenge	Page 7
06	The Approach & Delivery Lifecycle	Page 8
07	Code Spotlight	Page 9
08	Key Features & Role-Access Matrix	Page 10
09	Outcome & Engagement	Page 11

This document presents an in-depth look at the RouteOptim platform — its purpose, technical architecture, data model, the problems it solves, and the engineering decisions behind it. Every diagram, code snippet, and technical claim is grounded directly in the project source.

SECTION 01

Project Overview

What RouteOptim is, who it's for, and the shape of the system.

RouteOptim is an open-source, web-based delivery management and route-optimisation platform purpose-built for small and mid-sized businesses that need to coordinate fleets, drivers, customers, and deliveries without the cost — or operational overhead — of enterprise logistics software. It runs on a single Django 5 codebase, ships with role-based access for administrators, drivers, and customers, and exposes a public delivery-tracking portal that customers can use without an account.

The platform is built around five core entities — *Profile*, *Customer*, *Vehicle*, *Route*, and *Delivery* — wired together with foreign-key relationships that mirror how real delivery operations work: customers place orders, deliveries get assigned to vehicles, vehicles bundle into optimised routes, and every state transition is reflected in the data model. Each delivery is given an immutable UUID tracking ID so customers can look up status and rate the experience after completion.

Operators get a single dashboard for the day's work: a map view with live vehicle and delivery markers, CRUD on every entity, CSV import for bulk uploads, CSV export for reporting, and a one-click route optimiser that bundles pending deliveries onto a vehicle. Drivers see only their own queue. Customers see only their tracking link.

5

Core data entities

3

User roles

CRUD

On every entity

UUID

Public tracking

SECTION 02

System Architecture

Single Django process; external services kept thin and well-isolated.

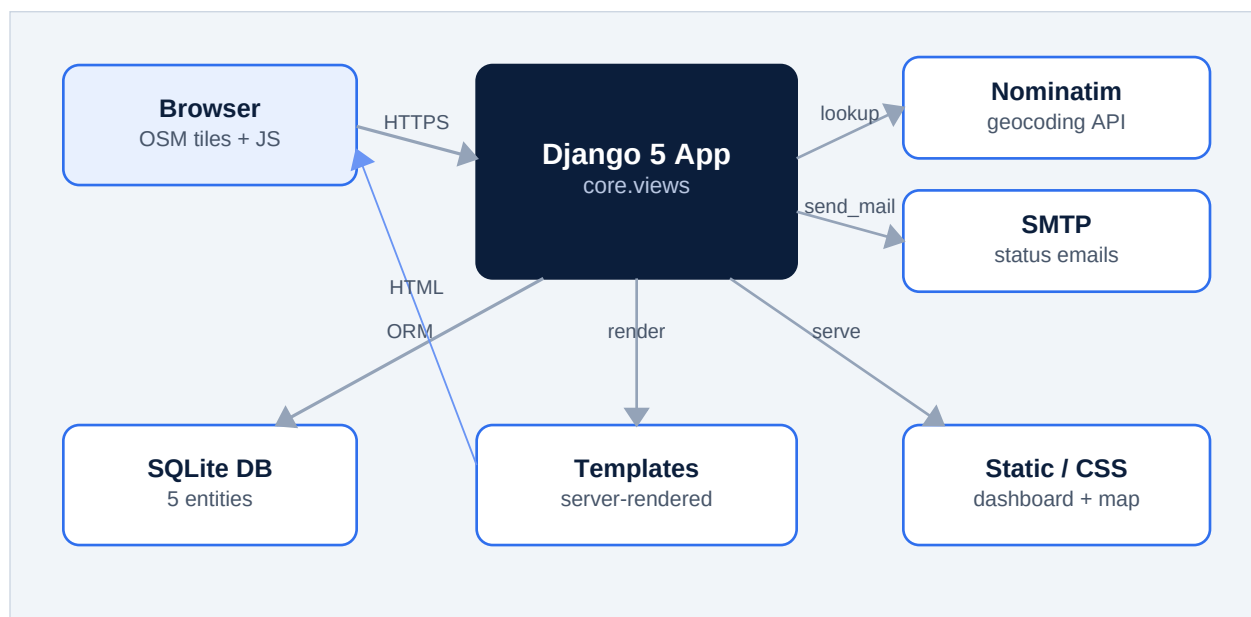


Figure — Browser ↔ Django ↔ SQLite, with Nominatim for geocoding and SMTP for status emails.

Every request enters the Django process, which fans out to four well-defined dependencies: the SQLite database for state, the templating layer for HTML, the Nominatim REST API for address → coordinates resolution, and the configured SMTP backend for status-change emails. OpenStreetMap tiles are loaded directly by the browser, keeping the Django path free of image-heavy responses. There are no microservices, no message queues, no Redis — by design, so the system can run on a single small VPS.

SECTION 03

Data Model & Entity Relationships

Five entities, one delivery-lifecycle audit trail, foreign keys enforced everywhere.

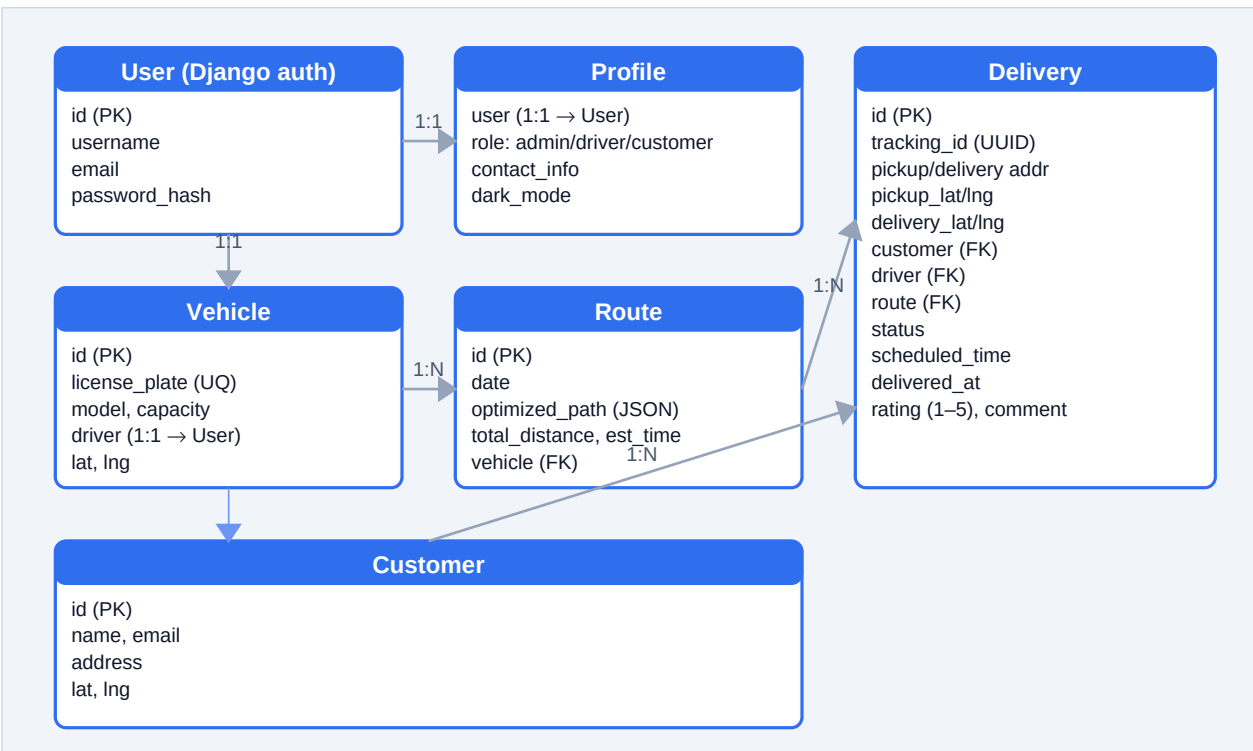


Figure — ERD — User & Profile (1:1), Vehicle to Routes (1:N), Route to Deliveries (1:N), Customer to Deliveries (1:N).

The model is the system: a *Customer* places a *Delivery*; the delivery is assigned to a *Vehicle* (and through it, a driver) on a given *Route*; each user has a *Profile* with a role that controls every protected view. Every *Delivery* carries an immutable *tracking_id* UUID, pickup/delivery coordinates for the map, a status (*pending* / *in-transit* / *delivered*), a scheduled time and a *delivered_at* timestamp, plus a customer rating from 1 to 5 with optional comment.

SECTION 04

Technology Stack

Stack chosen for single-host operation, clean reproducibility, and zero per-request API spend.

Layer	Technology & Details
Framework	Django 5.x — server-rendered templates, class-based views, built-in auth, sessions, messages
Language	Python 3 — single dependency declared in <i>requirements.txt</i> for clean reproducibility
Database	SQLite for development (swap to Postgres/MySQL via one settings change)
Frontend	Django templates + static CSS & JS — base layout, per-entity CRUD templates, dashboard, map view
Mapping	OpenStreetMap tiles in browser; Nominatim geocoding API with custom User-Agent and 5s timeout
Email	<i>django.core.mail.send_mail</i> — fires on status transitions when a customer email exists
Auth & Access	Django auth + <i>Profile.role</i> ; <i>RoleRequiredMixin</i> and <i>role_required</i> decorator on every protected view
Static Assets	Static directory with CSS, JS, and SVG logos

Why this stack. Django 5 collapses auth, templating, ORM, admin, sessions, and email into one well-tested framework. SQLite by default lets the application boot on any machine without an external service; the ORM-defined schema swaps to Postgres or MySQL by editing a single setting. OpenStreetMap + Nominatim keeps the mapping path royalty-free and avoids per-request API costs.

SECTION 05

The Challenge

Small businesses run real fleets without the budget or specialists for enterprise TMS software.

A small courier, a regional florist, a local meal-prep service, a parts distributor — each runs a fleet that is small enough to feel manageable on paper, but complex enough that managing it on spreadsheets quickly breaks down. Operators end up coordinating drivers over WhatsApp, exporting addresses into mapping apps one at a time, and learning about a missed delivery only when the customer calls. The cost of that ad-hoc coordination shows up as fuel waste, missed time windows, and customer churn.

Enterprise transportation-management systems exist, but they are priced for fleets ten times larger and require dedicated operators just to configure them. The brief for RouteOptim was a system that is enterprise in *shape* — a real data model, role-gated access, an audit-able delivery lifecycle, customer-facing tracking, structured reporting — but small-business friendly in *operation*: a single Django app that can be stood up in minutes, runs on a single host, and can be handed to a non-technical operator on day one.

Specifically the platform had to answer four operational questions every day, without the operator leaving the dashboard: **(1)** What deliveries are pending right now, and which are already in transit? **(2)** Which vehicles and drivers are available, and where are they? **(3)** Can these deliveries be bundled onto a single route, and who should drive it? **(4)** How do customers find out where their order is, and how do we capture their feedback after the fact?

SECTION 06

The Approach & Delivery Lifecycle

Data-first design, role-gated views, and an audit-able status flow from creation to rating.

Delivery lifecycle

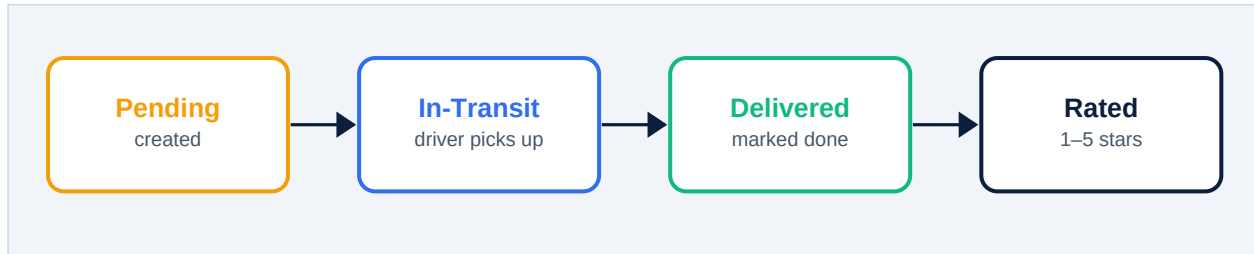


Figure — Each delivery moves through four states; status changes fire email notifications when an address is known.

Data model first

Five Django models express the entire business domain. Every relationship is enforced by foreign keys; reverse accessors are named for readability (*deliveries*, *routes*).

Role-based access, end to end

Every protected view inherits from **RoleRequiredMixin** (LoginRequiredMixin + UserPassesTestMixin) or is wrapped in the **role_required** decorator. The same templates render different sections based on *request.user.profile.role* — one UI, three clearly bounded views of it.

Geocoding, mapping, and public tracking

Addresses resolve to coordinates server-side via the Nominatim API with a custom User-Agent and a 5-second timeout that degrades gracefully. The map view aggregates active deliveries and vehicles into a single JSON markers payload that the browser renders over OpenStreetMap tiles. Every delivery gets a UUID *tracking_id*, exposed publicly at */track/<uuid>/*.

SECTION 07

Code Spotlight

Two short snippets that show how role-gated access and the route optimiser actually work.

Role-gated access — applied to every protected class-based view

core/decorators.py

```
class RoleRequiredMixin(LoginRequiredMixin, UserPassesTestMixin):
    allowed_roles = []

    def test_func(self):
        return (
            hasattr(self.request.user, 'profile')
            and self.request.user.profile.role in self.allowed_roles
        )
```

Route optimiser — bundles deliveries into a Route and assigns the driver in one action

core/views.py · excerpt

```
# core/views.py - optimize_route (admin only)
def optimize_route(request):
    vehicle = Vehicle.objects.get(id=vehicle_id)
    deliveries = Delivery.objects.filter(id__in=delivery_ids)
    ordered = list(deliveries.order_by('delivery_address'))

    path_data = json.dumps([
        { 'stop': i+1, 'address': d.delivery_address,
          'lat': d.delivery_lat, 'lng': d.delivery_lng }
        for i, d in enumerate(ordered)
    ])

    route = Route.objects.create(
        date=date.today(), optimized_path=path_data, vehicle=vehicle,
    )
    for d in ordered:
        d.route, d.driver, d.status = route, vehicle.driver, 'in-transit'
        d.save()
```

SECTION 08

Key Features & Role-Access Matrix

Eight production-grade capabilities, mapped to who can use them.

- › **Three-role authentication** — admin, driver, customer enforced by *RoleRequiredMixin* on every URL.
- › **Full CRUD on every entity** — class-based ListView/Detail/Create/Update/Delete with paginated lists and search on deliveries.
- › **Route optimiser** — bundles pending deliveries onto a vehicle with *optimised_path* JSON, ETA, and auto-driver assignment in one click.
- › **Geocoded map view** — JSON markers payload over OpenStreetMap tiles; Nominatim resolves addresses with proper headers.
- › **Public UUID tracking + ratings** — anyone with the link sees status; once delivered, a 1–5 star rating page unlocks.
- › **CSV import / export** — bulk-load deliveries with row-level error reporting; export deliveries and routes for back-office reporting.
- › **Email on status change** — Django *send_mail* notifies the customer on every transition and includes the rating link on completion.
- › **Operational dashboard** — status counts, 7-day daily trend, top customers, rolling average rating, served from a JSON chart endpoint.

	Admin	Driver	Customer
Dashboard	✓ full stats	✓ my queue	✓ my orders
Deliveries CRUD	✓	✓ own only	—
Vehicles CRUD	✓	view	—
Routes CRUD	✓	view	—
Optimise route	✓	—	—
CSV import / export	✓	—	—
Public tracking	✓	✓	✓
Rate delivery	—	—	✓ post-delivery

Figure — Role-access matrix — what admins, drivers, and customers can do.

SECTION 09

Outcome & Engagement

What the platform delivers, and how to take the conversation further.

RouteOptim turns the operational chaos of running a small fleet into a single, role-aware dashboard. Every entity has a place in the data model; every action by an admin, driver, or customer is captured, audit-able, and reflected in the views the others see. The route optimiser converts a list of pending deliveries into an assigned, in-transit route in one click; the public tracking portal eliminates the "where is my order?" phone calls; the ratings page closes the feedback loop without sending the customer to a third-party form.

Operationally, the codebase is structured for handover: one Django app (*core*), one settings module, a single requirements pin, ready-made templates for every CRUD operation, and a seed-data management command for demos. The same architecture extends naturally — swapping SQLite for Postgres, plugging in a real TSP/OR-tools solver inside *optimize_route*, or replacing Nominatim with a paid geocoder — without rewriting the rest of the system.

LET'S TALK

Mudasir Shafiq

Full-stack & data engineer

Email chmudasir.fr@gmail.com

Stacks Django • Python • React • Spring Boot • BigQuery • Docker